

PWM Generation for Motor/ESC Control Using STM32 and Input Button

This project demonstrates how to generate a PWM signal using an STM32 microcontroller to control a motor/ESC. An input push button is also used as a GPIO input.

The project uses the STM32 HAL library and STM32CubeMX/CubeIDE configuration.

1. STM32CubeMX / IOC Configuration

Select Timer

1. Open the `.ioc` file in STM32CubeMX or STM32CubeIDE.
2. Select the required timer.
3. Configure the clock source.

Clock Source → Internal Clock / External Clock

The clock source depends on the STM32 board and clock configuration being used.

Configure PWM Channel

Select the required timer channel:

Channel → PWM Generation CH1

For example:

TIM2 → Channel 1 → PWM Generation CH1

Configure Input Button

Configure one GPIO pin as an input for the push button.

Example:

Button Pin → PA15

The actual pin depends on the hardware design.

2. Timer Parameter Settings

For an ESC or servo-type PWM signal, a common PWM frequency is **50 Hz**.

The PWM period is:

$$T = 1 / f$$

Therefore:

$$\begin{aligned} T &= 1 / 50 \\ T &= 0.02 \text{ seconds} \\ T &= 20 \text{ ms} \\ T &= 20,000 \text{ } \mu\text{s} \end{aligned}$$

If the timer is configured with a timer tick of **1 μs** , the counter period is:

$$\begin{aligned} \text{Counter Period} &= 20,000 - 1 \\ &= 19,999 \end{aligned}$$

Therefore:

Prescaler	→ Timer Clock / 1 MHz - 1
Counter Mode	→ Up
Counter Period	→ 19999
Auto Reload	→ Enable
PWM Mode	→ PWM Mode 1
Pulse	→ Initial PWM pulse width

For example, an ESC may use:

1000 μs	→ Minimum throttle
1500 μs	→ Mid throttle
2000 μs	→ Maximum throttle

Note: The actual minimum and maximum pulse widths depend on the ESC. Some ESCs may require calibration before these values work correctly.

3. Input Button Configuration

Example button definition:

```
#define btn_port GPIOB
#define btn_pin  GPIO_PIN_4
```

The button can be read using:

```
HAL_GPIO_ReadPin(btn_port, btn_pin);
```

GPIO Logic Levels

STM32 HAL provides two macros for GPIO logic levels:

HAL Macro	Logic Level	Meaning
GPIO_PIN_RESET	0	LOW
GPIO_PIN_SET	1	HIGH

Whether `GPIO_PIN_RESET` means **button pressed** or **button released** depends on the button wiring and pull-up/pull-down configuration.

For example, with a pull-up configuration:

```
if (HAL_GPIO_ReadPin(btn_port, btn_pin) == GPIO_PIN_RESET)
{
    // Button is pressed
}
```

4. Start PWM Generation

The PWM output is started using:

```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
```

Where:

```
htim2      → Timer 2 handle  
TIM_CHANNEL_1 → Timer 2 Channel 1
```

The exact timer and channel depend on the configuration selected in STM32CubeMX.

5. Change PWM Pulse Width

The PWM pulse width can be changed using:

```
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, VALUE);
```

For example:

```
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 1000);
```

This generates approximately:

```
1000 µs pulse width
```

if the timer tick is configured to **1 µs**.

Example:

```
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 1500);
```

This generates approximately:

```
1500 µs pulse width
```

6. Motor/ESC PWM Variation Example

The following code gradually changes the PWM pulse width:

```
for (int i = 1000; i <= 2000; i += 100)  
{  
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, i);  
}
```

```
    HAL_Delay(3000);  
}
```

The PWM pulse width changes as follows:

```
1000 µs  
↓  
1100 µs  
↓  
1200 µs  
↓  
1300 µs  
↓  
1400 µs  
↓  
1500 µs  
↓  
1600 µs  
↓  
1700 µs  
↓  
1800 µs  
↓  
1900 µs  
↓  
2000 µs
```

Each PWM value is maintained for approximately **3 seconds**.

To return to minimum throttle:

```
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 1000);
```

7. Example Button-Controlled PWM

A basic implementation can be:

```
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);  
  
while (1)
```

```

{
    if (HAL_GPIO_ReadPin(btn_port, btn_pin) == GPIO_PIN_RESET)
    {
        for (int i = 1000; i <= 2000; i += 100)
        {
            __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, i);

            HAL_Delay(3000);
        }

        __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 1000);
    }
}

```

Warning: Do not use this code with a propeller attached to the motor during testing. Always perform initial ESC and motor testing with the propeller removed.

8. Verify GPIO Initialization

If the PWM or button does not work, check the generated STM32 HAL files.

Open:

```
stm32f4xx_hal_msp.c
```

Check whether the required timer peripheral and GPIO pins are initialized correctly.

Also verify:

- Timer clock is enabled.
- GPIO clock is enabled.
- PWM GPIO is configured with the correct Alternate Function.
- Button GPIO is configured as input.
- Correct timer channel is selected.
- Correct timer instance is being used.
- PWM output pin matches the physical hardware pin.

For example, the timer GPIO should be configured with the correct Alternate Function:

```

GPIO Mode → Alternate Function
Alternate Function → Correct TIMx_CHy

```

9. PWM Signal Summary

For a 50 Hz PWM signal:

```
Frequency = 50 Hz  
Period    = 20 ms  
Timer Tick = 1 µs  
ARR       = 19999
```

Typical ESC control:

```
1000 µs → Minimum throttle  
1500 µs → Mid throttle  
2000 µs → Maximum throttle
```

The PWM output is controlled by changing the timer's Compare Register:

```
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pulse_width);
```

Therefore:

```
STM32  
├── Timer Configuration  
├── PWM Generation  
├── Compare Value  
│   ├── 1000 µs → Low throttle  
│   ├── 1500 µs → Mid throttle  
│   └── 2000 µs → High throttle  
└── ESC → Motor
```

10. Key Concepts Learned

This project demonstrates:

- STM32CubeMX timer configuration

- PWM generation using STM32 timers
- 50 Hz PWM signal generation
- Timer prescaler configuration
- Auto-reload register configuration
- PWM Mode 1
- PWM pulse width control
- `HAL_TIM_PWM_Start()`
- `__HAL_TIM_SET_COMPARE()`
- GPIO input configuration
- Push-button input handling
- `GPIO_PIN_RESET` and `GPIO_PIN_SET`
- Basic ESC PWM control
- Debugging STM32 timer and GPIO initialization